SyncQuest

A Peer-to-Peer Collaborative Text Editor

Distributed Systems Project - Part 2: Synchronization Layer

Part 1 delivered serverless real-time editing with Yjs CRDTs over WebRTC. Part 2 layers three classic distributed-systems mechanisms on top without touching the document sync path: **vector-clock causality logging**, **Bully-algorithm coordinator election**, and a **SHA-256 hash-chain audit log**. This report covers the problem, concepts, design, implementation, usage, verification and trade-offs.

Stack	React + Vite, Yjs, y-webrtc, y-quill, Quill.js 1.3.7, y-indexeddb, Node.js (ws), Web Crypto API
Scope	Part 1 (core editor) and Part 2 (vector clocks, Bully election, hash-chain audit log)
Transport	WebRTC data channels (Google STUN); Yjs Awareness side-channel carries all Part 2 messages
Deployment	Signaling server runs standalone (Docker-ready); frontend is a static Vite build

1. Problem Statement

Part 1 showed that two or more browsers can edit one document in real time with no server owning it: Yjs's CRDT guarantees every replica converges, and WebRTC carries updates directly between browsers. That removes the central *document* authority, but it leaves several questions that any real distributed system must answer:

- **Causality.** Yjs converges correctly, but it does not tell a human *which edits were truly concurrent* and which were causally ordered. How can the system make that ordering observable?
- **Coordination.** Some jobs (e.g. deciding who records system events) need exactly one responsible peer. With no server, how do peers agree on one, and recover when it disappears?
- **Integrity.** If peers keep a history of what happened in the session, how can anyone detect that an entry was altered after the fact?

Part 2 answers these three questions with three well-known algorithms, implemented as small, separable modules so the already-tested Part 1 sync core is left unmodified.

2. Use Cases

- **Teaching Unit 3 (Synchronization):** a live, interactive demonstration of vector clocks and the Bully election algorithm, including failure and recovery.
- **Teaching Unit 4 (Emerging paradigms):** tamper-evident logging with a hash chain, and an explicit comparison with what a blockchain adds.
- **Session accountability:** a replicated record of who joined, who left, who coordinated, and periodic document checkpoints.
- **Fault-tolerance demos:** close the coordinator's tab and watch re-election; corrupt a log entry and watch verification flag it.

3. Topics Covered

Concept	Where it appears
CRDTs and Strong Eventual Consistency	Yjs Y.Text for the document body (Part 1; unchanged)
Peer-to-peer networking, NAT traversal	WebRTC data channels, STUN, SDP/ICE signaling (Part 1)
Logical time and causality	Vector clocks, happened-before partial order, concurrency detection (Part 2, Feature 1)
Election algorithms	Bully algorithm: ELECTION / ALIVE / COORDINATOR, timeouts, re-election on failure (Part 2, Feature 2)
Failure detection	Presence diffing via awareness; coordinator loss triggers re-election
Tamper-evident logs	SHA-256 hash chain, genesis block, chain verification (Part 2, Feature 3)
Centralisation trade-offs	Narrowly scoped coordinator vs. leaderless document; hash chain vs. blockchain

4. System Architecture

The system still has exactly two kinds of process: a browser tab running the React/Yjs frontend, and a minimal Node.js signaling relay that only forwards SDP/ICE handshake envelopes (`Map<room, Set<socket>>`; it never sees document content). Part 2 adds **no new process and no new server**: every new message rides the same WebRTC data channel as Part 1.

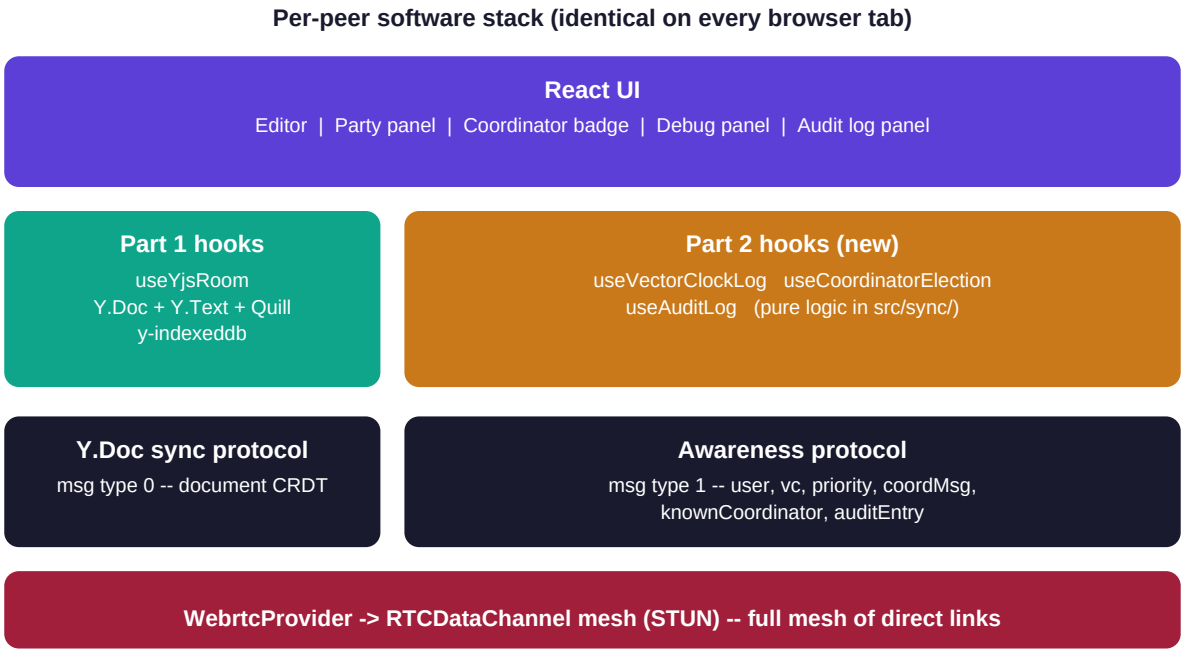


Figure 1 - Part 1 (teal) and Part 2 (amber) share one WebRTC mesh but use different protocol messages: documents on the Y.Doc sync protocol, all coordination metadata on the Awareness protocol.

4.1 Why the Awareness side-channel

y-webrtc multiplexes two message types over each data channel: type 0 (Y.Doc sync) and type 1 (Awareness). Awareness is last-value-wins, per-client, ephemeral state that is periodically re-broadcast as a keep-alive. All Part 2 data lives in extra fields of the local awareness state, so the CRDT merge path cannot be perturbed:

Awareness field	Owner feature	Contents
`user`	Part 1 presence	name, color, avatar, class, cursor
`vc`	Vector clocks	{ clock: {peerId: n}, ts } for the latest local edit
`priority`	Bully election	this peer's Y.Doc clientID (uint32)
`coordMsg`	Bully election	one-shot message: type, from, term, seq, (to)
`knownCoordinator`	Bully election	{ coordinatorId, term } - lets late joiners adopt the result
`auditEntry`	Audit log	{ entry, seq } - most recent chain entry from the coordinator

Because awareness re-broadcasts unchanged state, every consumer de-duplicates: vector clocks by timestamp, election and audit messages by a per-sender monotonically increasing `seq`.

4.2 Remote vs. local edits

y-webrtc applies every remote update with `Y.applyUpdate(doc, update, room)`, so a Y.Doc `update` event's *origin* equals the provider's Room for remote edits and something else for local ones.

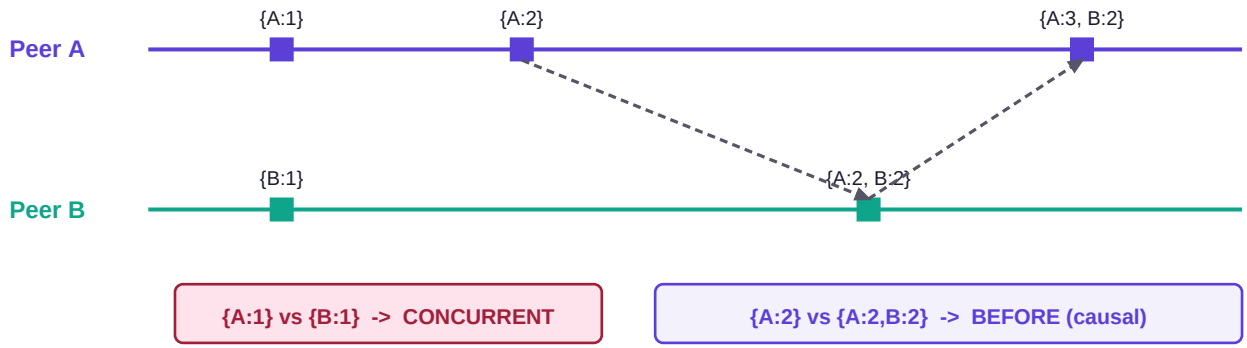
`sync/originUtils.js` exposes this as `isRemoteUpdateOrigin()`; the vector-clock hook uses it to tick its own clock only for edits the local user actually made.

5. Feature 1 - Vector Clock Logging

Files: `sync/vectorClock.js` (pure functions), `hooks/useVectorClockLog.js`, `components/DebugPanel.jsx` (Vector Clocks tab).

A vector clock is a map `{ peerId: counter }`, with peerId the string form of the Y.Doc clientID. Rules implemented:

- **Local edit:** increment own entry, broadcast the new clock (and a timestamp) in awareness field `vc`.
- **Receive:** merge by component-wise maximum (`mergeClocks`), then log the entry.
- **Compare** (`compareVectorClocks`): *before* if less-or-equal everywhere and strictly less somewhere; *after* symmetrically; *equal* if identical; otherwise *concurrent* - neither event could have influenced the other.



Dashed arrows: awareness messages carrying a clock; the receiver merges by component-wise max.

Figure 2 - Peer A's {A:1} and Peer B's {B:1} are incomparable (concurrent). After B has seen A's second edit, its next clock {A:2,B:2} dominates {A:2}, so the order is causal.

The debug panel keeps a rolling log (60 entries) and labels each entry with its relation to the previously shown entry, colouring concurrent ones pink with a "concurrent" badge. This is genuine vector-clock math on real edit events, not canned data.

> **Relationship to Yjs.** Yjs already resolves concurrent inserts deterministically using its own (clientID, clock) tags. Vector clocks here do not change that; they expose the causal structure Yjs hides, which makes the distinction between *conflict resolution* and *causality tracking* visible.

6. Feature 2 - Bully Coordinator Election

Files: `hooks/useCoordinatorElection.js`, `components/CoordinatorBadge.jsx`, `components/DebugPanel.jsx` (Election Log tab).

Every peer has a **priority**: its Y.Doc clientID, a random uint32 Yjs already assigns per tab. This was chosen over a join-time timestamp deliberately - unsynchronised wall clocks are an unreliable tie-breaker in a distributed system. The highest priority wins.

Message	Meaning	Handling
ELECTION	Sender suspects no coordinator; asks higher peers to object	Higher-priority receiver replies ALIVE and starts its own election
ALIVE (to X)	A higher-priority peer is up	Addressee cancels its victory timeout and waits for COORDINATOR
COORDINATOR	Sender has won	Every receiver records the sender as coordinator

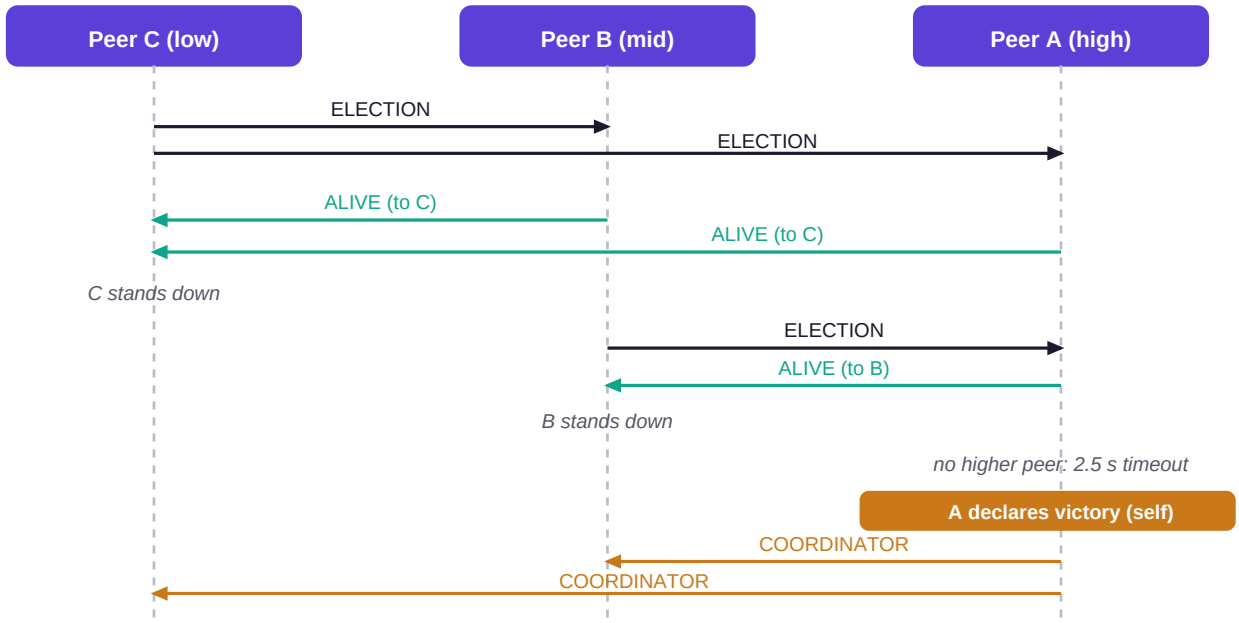


Figure 3 - Election among three peers. Lower peers broadcast ELECTION; higher peers answer ALIVE and take over; the highest peer times out waiting for an objection (2.5 s) and announces COORDINATOR.

6.1 Triggers and timing

- **Startup:** 1.5 s after the mesh settles, a peer adopts any `knownCoordinator` already in the replicated awareness state; if none, it starts an election.
- **Coordinator loss:** the hook diffs the presence list (same technique as the toast notifications); if the current coordinator vanishes, it clears the role and starts an election.
- **Manual:** the "Force re-election" button exercises the protocol while the coordinator is alive. Bully correctly re-confirms the existing highest peer.
- **Guard:** an already-elected coordinator ignores a stray late ELECTION from a lower peer instead of re-electing itself (a real bug found and fixed in testing).

6.2 Why Bully rather than Ring

Bully assumes any peer can message any other directly, which the WebRTC full mesh from Part 1 already provides. Ring election would require building and maintaining a logical ring over a network that has none. Messages are broadcast and filtered by priority instead of being literal point-to-point sends - behaviourally equivalent on a fully connected topology, and noted as a simplification.

7. Feature 3 - Hash-Chain Audit Log

Files: `sync/hashChain.js`, `hooks/useAuditLog.js`, `components/AuditLogPanel.jsx`.

The audit log is an append-only list of significant system events. Each entry stores `index, timestamp, eventType, data, prevHash, hash`, where the hash covers all of the other fields plus the previous entry's hash. Hashing uses the browser's native Web Crypto `crypto.subtle.digest('SHA-256')`, so no crypto library is added.

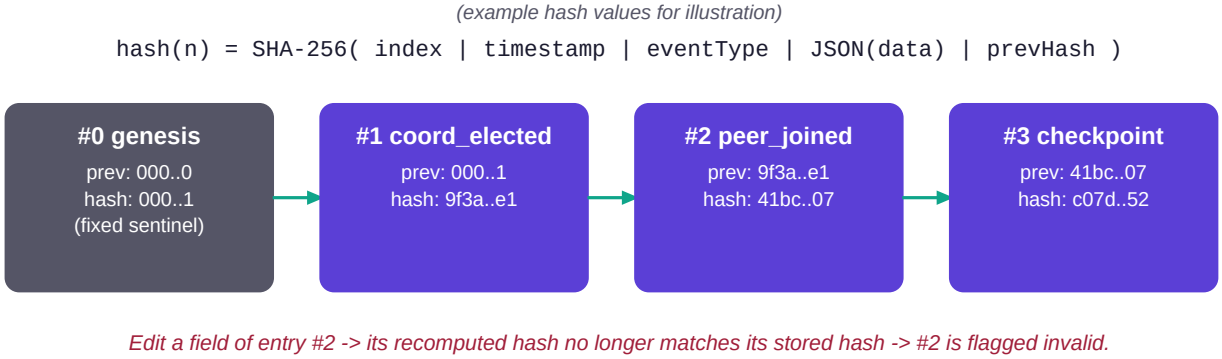


Figure 4 - Each entry commits to its predecessor. Altering an entry's fields makes its stored hash stop matching, so that entry is flagged. An edit that also recomputed the hash would instead break the link at the next entry.

Event type	Written by coordinator when...
`coordinator_elected`	this peer becomes coordinator
`peer_joined` / `peer_left`	the presence list gains or loses a peer
`checkpoint`	every 5 document updates, or every 45 s if any edits are pending (stores text length)

7.1 Replication

- Only the **current coordinator appends**. It broadcasts each new entry in the `auditEntry` awareness field.
- Receivers accept an entry only from the peer they currently believe is coordinator, and only if `entry.index` equals their chain length and `entry.prevHash` equals their tail hash; otherwise it is skipped with a console warning.
- The genesis entry is a fixed sentinel identical on all peers, so chain roots match with no network round-trip.
- The chain lives in each peer's React state - no server stores it.

7.2 Verification and the tamper demo

`verifyChain` recomputes every entry's hash and checks its `prevHash` link, shown as a green check or red cross per entry. The "Tamper" button mutates one entry's data locally (never broadcast). Only that entry turns invalid; the others stay valid, as confirmed by running the project's verifyChain (see the Technical Deep Dive, section 8.4). A tamper that also recomputed the hash would break the next link instead.

Why a hash chain and not a blockchain? Blockchain consensus (mining, longest-chain rules) exists so mutually untrusting writers can agree on one order. Here there is a single writer at a time, so the only property needed is tamper-evidence, which a plain hash chain provides with no mining delay or

fork handling.

8. Coordinator Authority vs. "No Central Authority"

Part 1 claimed the system has no central authority over the document. Part 2 introduces a coordinator, so the tension must be addressed honestly:

- The coordinator's authority is deliberately narrow: it decides only *who appends the next audit entry*. It never orders, validates or relays document edits.
- If it disconnects, typing continues unaffected (Part 1 path untouched). A new coordinator is elected within seconds; only audit-log growth pauses in the gap.
- Concurrent appends by several peers would need their own conflict resolution (for example per-peer sub-chains merged later); a single writer was judged the right complexity trade-off.

9. Gamified Interface (SyncQuest)

The "realm and party" skin from Part 1 is retained: room = Realm, share link = Portal Link, connection status = Portal state, presence = Party panel (random avatar, name and class), XP bar derived from the shared Y.Text length, toast notifications, and a dark neon theme. Part 2 adds:

- a persistent **Coordinator badge** (crown icon, "YOU" tag on the coordinator itself, "Electing coordinator..." state, force re-election button);
- a toggleable **Debug panel** (bottom-right) with Vector Clocks and Election Log tabs;
- a toggleable **Audit log panel** (bottom-left) showing truncated hash, previous hash and live validity of every entry, plus the tamper button.

10. Implementation Map

File	Responsibility
`signaling-server/server.js`	WebSocket relay of SDP/ICE only; room map; Dockerfile provided
`hooks/useYjsRoom.js`	Creates Y.Doc, IndexedDB persistence, WebrtcProvider, awareness identity; connection status with 8 s error heuristic
`components/Editor.jsx`	Quill editor bound to Y.Text via y-quill, remote cursors
`sync/vectorClock.js`	create / clone / increment / merge / compare vector clocks (pure)
`sync/originUtils.js`	Distinguishes remote from local Y.Doc updates
`sync/hashChain.js`	Genesis entry, SHA-256 entry hashing, chain verification
`hooks/useVectorClockLog.js`	Per-peer clocks, awareness broadcast, rolling log
`hooks/useCoordinatorElection.js`	Bully state machine, timeouts, startup adoption, re-election
`hooks/useAuditLog.js`	Coordinator-only appends, replication, periodic checkpoints, verification, tamper
`App.jsx`	Wires all hooks together; Part 2 hooks sit beside, not inside, the Part 1 sync path

11. How to Run the Project

Prerequisite: Node.js 18 or newer.

```
cd p2p-editor
npm run install:all     # installs signaling-server + frontend
npm run signaling       # Terminal 1: ws://localhost:4444
npm run dev             # Terminal 2: http://localhost:5173
```

Open `http://localhost:5173`; a room code is generated and appended as `?room=...`. Open the same URL in 2-4 tabs (use an incognito window for a distinct identity). For two real devices on one Wi-Fi, set `VITE_SIGNALING_URL=ws://<LAN-IP>:4444` in `frontend/.env`, restart the dev server, and open `http://<LAN-IP>:5173` on both devices (allow the firewall prompt).

12. How to Demo Part 2

Goal	Steps	Expected result
Concurrent edits	Two tabs; click into the editor in both; type in the same region within ~1 s of each other	Entries highlighted pink with a concurrent badge in Vector Clocks tab
Causal ordering	Type in tab 1, wait ~2 s, then type in tab 2	Entries show before / after instead of concurrent
Automatic election	Open 2-3 tabs on one room	All badges converge on the same coordinator within ~1.5 s of mesh settling
Re-election	Close the tab shown as coordinator	Remaining badges show Electing..., then a new coordinator, with the reason visible in the Election Log
Forced election	Click Force re-election while the coordinator is alive	ELECTION/ALIVE exchange; coordinator unchanged
Audit growth	Join a 4th tab; type a few edits	peer_joined and checkpoint entries appear in the existing peers' logs (a newly joined tab may not receive earlier entries)
Tamper test	Click Tamper on a non-genesis entry	The clicked entry turns invalid; the other entries stay valid

13. Testing and Verification Summary

From the project's test notes: the production build passes (`npm run build`), and the scenarios below were run end-to-end with 3-4 real browser tabs using automated browser control against the running app, not a mock.

Check	Result
Part 1: two-tab and three-tab real-time sync	All tabs converge to identical text within ~1 s
Part 1: presence on tab close; refresh recovery	Party count drops within seconds; local content restored, then re-syncs
Part 1: signaling server killed after connect	Already-connected peers keep syncing
Automatic coordinator election at startup	All tabs agree on one coordinator without manual action
Genuinely concurrent edits (true-parallel typing in two tabs)	Flagged as concurrent by vector clock comparison
Re-election after closing the coordinator's tab	New coordinator elected in well under a second once the closing tab's beforeunload cleanup fires
Forced re-election with coordinator alive	No-op but correct: existing coordinator confirmed
Fourth peer joins mid-session	peer_joined entry appended to the audit chain
Tamper test	Tampered entry flagged invalid; other entries unaffected (confirmed by running verifyChain on a test chain)

Bug found and fixed: an already-elected coordinator could be pushed back into re-electing itself by a late ELECTION message from a lower-priority peer. The handler now ignores such messages when this peer is the settled coordinator.

14. Known Limitations

- **No audit-chain resync.** If a peer misses a coordinator broadcast, it logs a warning and stops appending until reload; no backfill request exists.
- **Bully over broadcast.** Targeted ELECTION/ALIVE is implemented as broadcast plus priority filtering - equivalent on a full mesh, but a simplification.
- **Awareness is last-value-wins.** A peer's state holds only the latest value of each field, so a peer that was absent or reconnecting sees only the most recent message or audit entry, not the earlier ones.
- **Tamper button is local-only.** It demonstrates detection; it does not model a malicious peer rewriting the shared log.
- **No TURN server.** STUN-only NAT traversal; restrictive or symmetric NATs and client-isolated Wi-Fi may block direct connections.
- **Error status is heuristic.** The "Connection error" badge is an ~8 s timeout, since y-webrtc exposes no single handshake-failed event.
- **Trust model.** Peers trust whichever peer they believe is coordinator; there is no signature scheme, so a hostile coordinator could write false entries (the chain proves integrity, not authenticity).
- **Environment note.** If `frontend/.env` still points at an old LAN IP, same-machine tests show a harmless WebSocket timeout (same-browser tabs sync via BroadcastChannel).

15. Possible Future Work

- Digitally sign audit entries per peer to add authenticity to integrity.
- Chain resync: a peer that detects a gap asks the coordinator for missing entries.
- Per-peer sub-chains merged by vector clock, removing the single-writer restriction.
- A TURN fallback and QR-code share links.
- Feed the vector-clock log from document operations at finer granularity (per Yjs transaction rather than per update event).

16. Summary

SyncQuest now demonstrates, in one working app, the full arc from Part 1's serverless collaborative editing to Part 2's coordination layer. **Yjs CRDTs** still resolve document conflicts without arbitration; **vector clocks** make the underlying causal ordering visible and reliably flag truly concurrent edits; the **Bully algorithm** elects and re-elects a single coordinator over the existing WebRTC mesh; and a **SHA-256 hash chain** gives that coordinator a tamper-evident audit log that the connected peers replicate.

All Part 2 traffic travels on Yjs's Awareness side-channel, so the proven Part 1 sync path was left untouched, and the coordinator's authority was scoped to audit-log writing only, keeping the document itself leaderless. Known gaps, notably audit-chain resync and the lack of entry signatures, are documented rather than hidden.

See README.md for setup and testing detail, ARCHITECTURE.md for Part 1 data-flow diagrams, and the Technical Deep Dive and Syllabus Correlation documents for deeper material.